

Отчёт по лабораторной работе №2

Математические основы защиты информации и информационной безопасности

Ли Хан

Содержание

1	Цель работы	5
2	Анализ выполненных упражнений	6
2.1	Анализ алгоритма маршрутного шифрования	6
2.2	шаг	6
2.3	маршрутного шифрования компилятора	7
2.4	результат	7
2.5	Анализ алгоритма шифрования с помощью решеток	8
2.6	шаг	8
2.7	шифрования с помощью решеток компилятора	9
2.8	результат	9
2.9	Анализ шифра Виженера	10
2.10	шаг	10
2.11	шифра Виженера компилятора	11
2.12	результат	11
3	Вывод	12

Список иллюстраций

2.1	маршрутного шифрования компилятора	7
2.2	маршрутного шифрования результат	7
2.3	шифрования с помощью решеток компилятора	9
2.4	шифрования с помощью решеток результат	9
2.5	шифра Виженера компилятора	11
2.6	шифра Виженера результат	11

Список таблиц

1 Цель работы

Изучить основные принципы шифров перестановки, программно реализовать маршрутное шифрование, шифрование с помощью решеток и шифр Виженера, а также понять логический процесс изменения порядка символов или их замены.

2 Анализ выполненных упражнений

2.1 Анализ алгоритма маршрутного шифрования

Логика алгоритма: Алгоритм изменяет порядок символов текста с помощью матричной перестановки. Текст записывается по строкам в матрицу, ширина которой зависит от длины ключа, а затем считывается по столбцам в порядке, определяемом алфавитным весом букв ключа.

2.2 шаг

- Определение матрицы: Если текст состоит из 30 символов, а длина ключа — 6, получается матрица 5×6 .
- Запись по строкам: Писываем текст горизонтально, как при обычном письме.
- Сортировка по ключу: Каждой букве ключа (например, «ПАРОЛЬ») присваивается номер согласно её месту в алфавите ($A=1$, $L=2$, $O=3...$).
- Чтение по столбцам: Считываем буквы не по порядку, а согласно номерам букв ключа. Соседние в тексте буквы оказываются в разных строках, и связь между ними разрывается.

2.3 маршрутного шифрования компилятора

```
def columnar_transposition_encrypt(text, key):
    # 中文: 清理文本, 将空格移除并统一为大写
    # RU: Очистка текста: удаление пробелов
    text = text.replace(" ", "").upper()

    # 中文: 如果文本长度不是密钥长度的倍数, 用 "A" 填充
    # RU: Если длина текста не кратна длине ключа
    n = len(key)
    while len(text) % n != 0:
        text += 'A'

    # 中文: 根据密钥字母的字母表顺序确定读取列的索引
    # RU: Определяем порядок чтения столбцов
    key_order = sorted(range(len(key)), key=lambda k: key[k])

    # 中文: 将文本分割成矩阵行
    # RU: Разделяем текст на строки матрицы
    rows = [text[i:i + n] for i in range(0, len(text), n)]

    # 中文: 按照计算出的列顺序提取字符, 形成密文
    # RU: Извлекаем символы в соответствии с порядком
    ciphertext = ""
    for col_idx in key_order:
        for row in rows:
            ciphertext += row[col_idx]

    return ciphertext
```

Рис. 2.1: маршрутного шифрования компилятора

2.4 результат

```
# 示例: 密码由 Франсуа Виет 开发
# Пример: Шифр, разработанный Франсуа Виетом
print(f"路线加密结果: {columnar_transposition_encrypt('нельзя недооценивать противника', 'пароль')}")
路线加密结果: ЕЕНПНЗОАТАЬОВКННЕЬВЛДИРИЯЦТИА
```

Рис. 2.2: маршрутного шифрования результат

2.5 Анализ алгоритма шифрования с помощью решеток

Логика алгоритма: Этот метод имитирует вращение физической решетки с прорезями. В матрице $2k \times 2k$ задаются определенные координаты отверстий. После каждого заполнения группы символов координаты поворачиваются на 90 градусов по часовой стрелке до полного заполнения всех четырех направлений.

2.6 шаг

- Конструкция решетки: В квадрате 4×4 вырезаются 4 отверстия. Они выбираются так, чтобы при повороте карточки на 90, 180 и 270 градусов отверстия никогда не накладывались на уже заполненные клетки.
- В исходном положении через 4 отверстия вписываются первые 4 буквы.
- Решетка поворачивается на 90 градусов, открывая новые пустые клетки, куда вписываются следующие 4 буквы.
- Процесс повторяется, пока все 16 клеток не будут заполнены.

2.7 шифрования с помощью решеток компилятора

```
import numpy as np

def grille_cipher_encrypt(text, k=2):
    # 中文: 初始化 2k x 2k 的矩阵, 并准备字符列表
    # RU: Инициализируем матрицу 2k x 2k
    size = 2 * k
    grid = np.full((size, size), ' ', dtype=str)
    chars = list(text.replace(" ", "").upper())

    # 中文: 定义初始孔位坐标 (示例 4x4 栅格)
    # RU: Определяем начальные координаты
    holes = [(0, 0), (0, 3), (2, 1), (3, 2)]

    # 中文: 执行四次顺时针 90 度旋转, 每次在孔位填入字符
    # RU: Выполняем четыре поворота на
    for _ in range(4):
        for r, c in holes:
            if chars:
                grid[r, c] = chars.pop(0)
        # 核心逻辑: 计算旋转 90 度后的新孔位坐标
        # RU: Логика: вычисление новых
        holes = [(c, size - 1 - r) for r, c in holes]

    # 中文: 按列读取矩阵中的所有字母生成密文
    # RU: Считываем все буквы из матрицы
    return "".join(grid.T.flatten())
```

Рис. 2.3: шифрования с помощью решеток компилятора

2.8 результат

```
print(f"栅格加密结果: {grille_cipher_encrypt('Д О Г О В О Р П О Д П И С А Л И', 2)}")
```

栅格加密结果: А П С И Р Г П Л О В И О

Рис. 2.4: шифрования с помощью решеток результат

2.9 Анализ шифра Виженера

Логика алгоритма: Это многоалфавитный шифр подстановки. Программа использует зацикленный ключ и выполняет циклический сдвиг для каждого символа по формуле (индекс текста + индекс ключа) % длина алфавита.

2.10 шаг

- Улучшение шифра Цезаря: В шифре Цезаря все буквы сдвигаются на k позиций. В шифре Виженера каждая буква сдвигается на разное расстояние, зависящее от буквы ключа.
- Математическая формула:

$$C_i = (P_i + K_i) \pmod{n}$$

Где C — шифртекст, P — открытый текст, K — позиция буквы ключа в алфавите, n — размер алфавита.

- Логика: Если буква ключа — «В» (2-я в алфавите), то соответствующая буква текста сдвигается на 2 позиции вперед. Поскольку ключ циклический, постоянное смещение делает невозможным простой частотный анализ.

2.11 шифра Виженера компилятора

```
def vigenere_encrypt(text, key):
    # 中文: 定义完整的俄语字母表 (33个字母)
    # RU: Определяем полный русский алфавит (33 б
    alphabet = "АБВГДЕЁЖЗИЙКЛМНОПРСТУФХЦЧШЩЪЫЬЭЮЯ"
    text = text.upper().replace(" ", "")
    key = key.upper()

    ciphertext = ""
    for i in range(len(text)):
        if text[i] in alphabet:
            # 中文: 计算当前字符和对应密钥字符在字母表中的索引
            # RU: Находим индексы текущего симво
            p_idx = alphabet.index(text[i])
            k_idx = alphabet.index(key[i % len(key)])

            # 中文: 执行循环位移: (明文索引 + 密钥索引) % 33
            # RU: Выполняем циклический сдвиг: (и
            c_idx = (p_idx + k_idx) % len(alphabet)
            ciphertext += alphabet[c_idx]

    return ciphertext

# 示例: 使用 "МАТЕМАТИКА" 作为密码
# Пример: использование слова «МАТЕМАТИКА» в ка
```

Рис. 2.5: шифра Виженера компилятора

2.12 результат

```
print(f"维吉尼亚加密结果: {vigenere_encrypt('криптография', 'математика')}")
```

维吉尼亚加密结果: ЧРЫФЯОХШКФХЯ

Рис. 2.6: шифра Виженера результат

3 Вывод

В ходе лабораторной работы были успешно реализованы три классические системы шифрования. Эксперимент подтвердил, что шифры перестановки, не меняя содержания символов, эффективно скрывают логику открытого текста через перестановку позиций и циклические сдвиги.